

Labo 04 — Réponses commentées

Question 1 — `COPY ../commun/config.yml`

Réponse. Le fichier est **hors du build context**. Le client n'a empaqueté que `~/projets/api/` : `../commun/` n'a jamais été transmis au daemon, qui ne peut donc pas le copier. Solution correcte : élargir le contexte et adapter le chemin — `docker build -f api/Dockerfile -t api:1.0 ~/projets` avec `COPY commun/config.yml /app/`.

Pourquoi. `COPY` n'est pas un `cp` : il opère sur l'archive reçue par le daemon, pas sur votre disque. Un chemin sortant du contexte ne désigne rien.

Nuance. C'est bien pour cela que `-f`, un chemin absolu ou `sudo` sont sans effet : le problème n'est pas une permission ni une localisation du Dockerfile, mais l'**absence physique** du fichier là où le build s'exécute. Attention en élargissant le contexte : on transfère alors tout `~/projets`, donc un `.dockerignore` adapté devient indispensable. La solution vraiment propre en entreprise est de publier `commun` comme dépendance (artefact Maven, paquet npm) plutôt que de le copier entre projets.

Exemple.

```
docker build -t x .
# ERROR: failed to solve: failed to compute cache key:
#   failed to calculate checksum of ref ...: "/hors.txt": not found
```

Question 2 — 3 min 20 s de `transferring context`

Réponse. Le client empaquette et envoie **tout** le dossier au daemon : 1,1 Go de `node_modules/` et `.git/` inutiles. Correction : un `.dockerignore` à la racine du contexte.

```
node_modules
.git
dist
*.log
.env*
```

Le second risque éliminé est celui de la **fuite de données** : sans exclusion, un `COPY . .` embarque `.git` (donc tout l'historique, y compris des secrets supprimés depuis) et les fichiers `.env` locaux **dans l'image publiée**.

Pourquoi. Le contexte est transféré avant même la première instruction. Ni le cache ni le multi-stage n'y changent quoi que ce soit : `.dockerignore` est le seul levier.

Nuance. Copier `node_modules` est aussi **fonctionnellement dangereux** : les modules compilés pour votre poste (macOS/arm64) peuvent être incompatibles avec l'image Linux. La règle est d'installer les dépendances **dans** l'image (`npm ci`), jamais de les copier.

Exemple.

```
du -sh .git node_modules
docker build -t web:1.0 .      # comparez la ligne "transferring context" avant/après
```

Question 3 — EXPOSE 8080 et curl qui ne répond pas

Réponse. Non, l'image n'est pas cassée. EXPOSE ne publie aucun port : c'est une métadonnée déclarative. Il faut `docker run -d -p 8080:8080 mon-api:1.0`.

Pourquoi. Le conteneur a sa propre pile réseau (namespace `net`). Son port 8080 n'est pas celui de l'hôte. Seul `-p` crée la règle de redirection de l'hôte vers le conteneur.

Nuance. EXPOSE n'est pas inutile pour autant : il documente l'image (`docker image inspect` le montre), il est lu par `docker run -P` qui publie alors tous les ports déclarés sur des ports libres aléatoires de l'hôte, et Compose s'en sert. Mais ce n'est jamais une condition : on peut parfaitement publier un port jamais déclaré. Retenez : EXPOSE = documentation, `-p` = mécanisme.

Exemple.

```
docker run -d --name web nginx:alpine
curl -s localhost:80 | head -1          # rien : aucun port publié
docker rm -f web
docker run -d --name web -p 8080:80 nginx:alpine
curl -s localhost:8080 | head -1      # <!DOCTYPE html>
docker rm -f web
```

Question 4 — RUN contre CMD

Réponse. B est correct.

- **A** exécute l'application **pendant le build**. Deux issues possibles : soit l'application démarre et ne rend jamais la main, et le build reste bloqué indéfiniment ; soit elle échoue, et le build échoue avec elle. Dans tous les cas, l'image finale n'a **aucune** commande de démarrage : `docker run` héritera du `CMD` de l'image de base.
- **B** enregistre la commande dans les métadonnées, sans rien exécuter au build. Elle sera lancée à chaque `docker run`.

Pourquoi. RUN sert à **fabriquer** l'image (installer, compiler, créer un utilisateur) ; `CMD` / `ENTRYPOINT` décrivent ce qui se passera **plus tard**, à l'exécution.

Nuance. L'erreur est plus facile à commettre qu'il n'y paraît avec des commandes ambiguës comme `RUN npm start` ou `RUN python app.py`. Un bon indice : si l'étape de build ne se termine jamais, c'est presque toujours un `RUN` qui lance un serveur.

Exemple.

```
docker image inspect --format '{{json .Config.Cmd}}' img-A  # ["/bin/sh"] hérité de la base
docker image inspect --format '{{json .Config.Cmd}}' img-B  # ["java", "-jar", "/app/api.jar"]
```

Question 5 — ENTRYPOINT + CMD contre CMD seul

Réponse.

	<code>docker run img</code>	<code>docker run img --debug</code>
A	<code>java -jar /app/api.jar --spring.profiles.active=prod</code>	<code>java -jar /app/api.jar --debug</code> (le <code>CMD</code> est remplacé, l' <code>ENTRYPOINT</code> reste)
B	<code>java -jar /app/api.jar --spring.profiles.active=prod</code>	<code>--debug</code> — Docker tente d'exécuter <code>--debug</code> comme une commande, et échoue

Seule **B** permet `docker run img sh`, puisque tout le `CMD` est remplaçable. Avec **A**, `docker run img sh` passerait `sh` en argument à `java`. Il faut alors `docker run --entrypoint sh -it img`.

Pourquoi. Les arguments de `docker run` **remplacent** le `CMD` et sont **ajoutés** à l'`ENTRYPOINT`.

Nuance. Le motif A est celui des images d'application, et le comportement de B avec `--debug` (exec: "--debug": executable file not found) est un symptôme fréquent chez ceux qui n'ont défini qu'un `CMD`. En pratique, A + `--entrypoint` pour le débogage est la bonne combinaison en entreprise : l'image a un rôle clair, et le contournement reste possible.

Exemple.

```
docker run --rm --entrypoint sh -it mon-api:1.0    # obtenir un shell malgré l'ENTRYPOINT
docker image inspect --format 'EP={{json .Config.Entrypoint}} CMD={{json .Config.Cmd}}' mon-api:1.0
```

Question 6 — `CMD java -jar` en forme shell

Réponse. La forme *shell* fait exécuter `/bin/sh -c "java -jar /app/api.jar"`. Sur une image de base Debian ou Ubuntu — le cas de `eclipse-temurin:21-jre` — le PID 1 est alors `sh`, pas Java. `sh` ne relaie pas `SIGTERM` : Java ne le reçoit jamais, ses *shutdown hooks* ne s'exécutent pas, Docker attend son délai de grâce de 10 secondes puis envoie `SIGKILL`. Correction :

```
ENTRYPOINT ["java", "-jar", "/app/api.jar"]
```

Pourquoi. Une ligne de syntaxe décide de l'identité du PID 1, et le PID 1 est le seul destinataire des signaux de Docker.

Nuance — et elle est importante. Le même Dockerfile sur une base **Alpine** ne produit pas le symptôme : le shell de busybox remplace son propre processus par la commande (exec implicite) quand celle-ci est simple, si bien que Java devient PID 1 et que l'arrêt est propre. Le comportement dépend donc de l'image de base, et c'est le pire des cas de figure : cela fonctionne sur le poste du développeur et casse en production. Il suffit par ailleurs d'ajouter un tube ou un `&&` pour que le shell reste, même sur Alpine.

Deux conséquences dépassent les 10 secondes. Les requêtes HTTP en cours sont coupées net à chaque déploiement — invisible en test, très visible en production. Et le code de sortie devient `137` au lieu de `143`, ce qui brouille la supervision : une alerte « conteneur tué » se déclenche à chaque déploiement normal, et l'équipe finit par l'ignorer — jusqu'au jour où elle signalait un vrai OOM.

Exemple.

```
docker inspect --format '{{json .Config.Entrypoint}}' mon-api:1.0
# ["/bin/sh","-c","java -jar /app/api.jar"] <- symptôme visible dans l'image
docker exec mon-api ps -o pid,args | head -3
# 1 /bin/sh -c java -jar /app/api.jar <- le shell est resté : arrêt cassé
# 7 java -jar /app/api.jar
```

Question 7 — `$JAVA_OPTS` dans une forme exec

Réponse. En forme exec, il n'y a **pas de shell** : personne n'interprète `$JAVA_OPTS`. La chaîne littérale `$JAVA_OPTS` est passée à Java comme argument, d'où l'erreur.

Deux corrections :

```
# 1. Un shell explicite : la variable est interprétée...
ENTRYPOINT ["sh","-c","exec java $JAVA_OPTS -jar /app/api.jar"]

# 2. Pas de variable du tout : valeurs écrites en dur
ENTRYPOINT ["java","-Xmx512m","-jar","/app/api.jar"]
```

Coût de chacune. La première réintroduit un shell comme PID 1 — d'où le `exec` dans la commande, indispensable pour que Java récupère le PID 1 et les signaux ; sans lui, on retombe sur le problème de la question 6. La seconde perd toute possibilité de régler la JVM par variable d'environnement au `docker run`, ce qui est gênant lorsqu'un même artefact doit tourner avec des réglages différents selon l'environnement.

Nuance. Pour la JVM, il existe une troisième voie, la meilleure : la variable `JAVA_TOOL_OPTIONS`, que la JVM lit **d'elle-même** au démarrage. On garde alors une forme exec pure, et le réglage reste possible à l'exécution : `docker run -e JAVA_TOOL_OPTIONS="-XX:MaxRAMPercentage=75" mon-api:1.0`.

Exemple.

```
# Démonstration de la non-substitution :
printf 'FROM alpine\nENV V="salut"\nENTRYPOINT ["echo","$V","fin"]\n' > Dockerfile
docker build -q -t t . && docker run --rm t # affiche : $V fin
```

Question 8 — Le mot de passe en `--build-arg`

Réponse. Non, il n'est pas en sécurité. La valeur du `--build-arg` apparaît en clair dans l'historique de l'image, que **toute** personne ayant accès à l'image peut lire.

Pourquoi. Chaque instruction est enregistrée dans les métadonnées avec les `ARG` actifs au moment de son exécution. La valeur transite aussi dans le cache de build, et dans les logs de la CI.

Nuance. Son raisonnement n'est pas complètement faux : `ARG` ne crée effectivement pas de variable d'environnement dans les conteneurs, contrairement à `ENV`. Mais « absent de l'environnement d'exécution » ≠ « absent de l'image ». Et si, comme c'est fréquent, il a écrit `ENV MDP=${DB_PASSWORD}`, la fuite est double : historique **et** environnement de tout conteneur. La vraie solution est le *secret mount* de BuildKit — `RUN --mount=type=secret,id=db cat /run/secrets/db` — qui n'inscrit rien dans aucune couche ; c'est le sujet du labo 08.

Exemple.

```
docker history --no-trunc argtest:1.0 --format '{{.CreatedBy}}' | head -4
# ARG DB_PASSWORD=Secr3t!
# ENV BUILT_WITH=Secr3t!
# RUN |1 DB_PASSWORD=Secr3t! /bin/sh -c echo ...
docker image inspect --format '{{json .Config.Env}}' argtest:1.0    # la fuite bis
```

Question 9 — Le build Maven de 6 minutes

Réponse.

```
WORKDIR /app
COPY pom.xml .
RUN mvn -q dependency:go-offline      # dépend UNIQUEMENT de pom.xml
COPY src ./src
RUN mvn -q package -DskipTests -o    # seule cette étape rejoue à chaque commit
```

Pourquoi. Le cache est invalidé par le **contenu** des fichiers copiés. Dans la version d'origine, `COPY . /app` change à chaque commit, donc l'étape suivante — le téléchargement de toutes les dépendances Maven, l'essentiel des 6 minutes — est systématiquement rejouée. En copiant d'abord le seul `pom.xml`, l'étape de dépendances n'est invalidée que lorsque les dépendances changent réellement.

Nuance. Restera lent : le build qui suit **toute modification du `pom.xml`** (ajout d'une dépendance, montée de version), et le premier build sur un agent de CI neuf, dont le cache est vide — c'est le cas le plus fréquent en entreprise, et la raison pour laquelle on configure un cache de build partagé (`--cache-from`, cache registry BuildKit) ou un miroir Nexus. Notez aussi que `dependency:go-offline` n'est pas parfait : certains plugins ne sont téléchargés qu'à l'exécution réelle du `package`.

Exemple.

```
docker build -t api:1.0 .          # 1er build : tout s'exécute
touch src/main/java/App.java
docker build -t api:1.1 .          # les étapes pom.xml affichent CACHED
```

Question 10 — Trois `RUN` `apt-get`

Réponse. Trois défauts distincts :

1. **Trois couches au lieu d'une**, et surtout un `rm` **inefficace** : la suppression a lieu dans une couche postérieure, elle masque les fichiers sans les retirer de l'image. Les ~40 Mo d'index apt sont toujours téléchargés avec l'image.
2. `apt-get update` **dans une couche séparée** : elle sera mise en cache durablement. Des semaines plus tard, l'`install` s'appuiera sur des index périmés et échouera en `404 Not Found` — le problème du *cache busting*.
3. `vim` **n'a rien à faire dans une image**, et l'absence de `--no-install-recommends` tire des dizaines de paquets superflus : poids, surface d'attaque, vulnérabilités à traiter.

```
RUN apt-get update \
&& apt-get install -y --no-install-recommends curl \
&& rm -rf /var/lib/apt/lists/*
```

Pourquoi. Une couche ne peut pas modifier la précédente : tout nettoyage doit se faire dans le même `RUN` que la création.

Nuance. Un outil de diagnostic dans une image de production est un choix, pas une faute en soi — mais il faut le décider. La tendance est inverse : images minimales (*distroless*), diagnostic par conteneur éphémère `docker debug` ou par un side-car. Sur Alpine, l'équivalent est `apk add --no-cache curl`, qui évite le nettoyage explicite.

Exemple.

```
docker history mon-image --format 'table {{.Size}}\t{{.CreatedBy}}' | head -5
```

Question 11 — Deux comportements, une règle

Réponse. La règle : **une instruction invalidée invalide toutes les suivantes**, et jamais les précédentes.

- Modifier le **dernier** `COPY` n'invalidé que lui et ce qui le suit : les huit premières étapes restent `CACHED`.
- Insérer un `ENV` en **troisième** position modifie la chaîne à partir de là. Toutes les instructions suivantes voient une couche parente différente : leur clé de cache change, donc tout est reconstruit.

Pourquoi. La clé de cache d'une étape est calculée à partir de l'identifiant de la couche parente **et** de l'instruction. Changer un maillon change tous les suivants.

Nuance. Le contre-intuitif est qu'une instruction à `0B` — un `ENV`, un `LABEL` — peut coûter dix minutes de build. D'où la pratique : les métadonnées volatiles (`LABEL` de version, date, commit Git) se placent **à la fin** du Dockerfile, jamais au début. Notez aussi que pour `COPY`, c'est le **contenu** qui compte : `touch` sur un fichier ne suffit pas à invalider le cache avec BuildKit, alors que le constructeur historique se basait sur les métadonnées du fichier.

Exemple.

```
docker build --progress=plain -t t . 2>&1 | grep -E 'CACHED|DONE'
```

Question 12 — `COPY` contre `ADD`

Réponse. `ADD` fait deux choses de plus : il **décompresse automatiquement** les archives locales reconnues (`.tar`, `.tar.gz` ...) dans la destination, et il sait **télécharger une URL**. La recommandation est d'utiliser `COPY` parce que ces comportements sont implicites et surprenants. Seul cas justifié : extraire une archive locale dans l'image, où `ADD` évite un `RUN tar`.

Pourquoi. Un `ADD monfichier.tar.gz /app/` ne copie pas le fichier : il le **déballe**. Sur un `ADD ./` générique, un `.tar` déposé par mégarde dans le contexte modifie silencieusement le contenu de l'image.

Nuance. Le téléchargement par URL est doublement à éviter : la ressource distante peut changer (build non reproductible), l'échec n'est pas toujours détecté, et surtout la signature n'est pas

vérifiée. La bonne pratique est `RUN curl -fsSL url -o f && echo "<sha256> f" | sha256sum -c -`, qui échoue si le contenu n'est pas celui attendu.

Exemple.

```
COPY target/api.jar /app/api.jar      # règle générale
ADD socle-outils.tar.gz /opt/         # cas légitime : extraction d'une archive locale
```

Question 13 — USER trop tôt

Réponse. À partir de `USER 1000:1000`, **toutes** les instructions suivantes s'exécutent sous cet utilisateur non privilégié : `apt-get install` (qui écrit dans `/usr`) et les `COPY` vers des dossiers appartenant à root échouent en `Permission denied`. `USER` doit être placé **le plus tard possible**, juste avant `CMD / ENTRYPOINT`.

Pourquoi. Les instructions du Dockerfile sont exécutées séquentiellement dans un conteneur temporaire, avec l'identité courante. `USER` change cette identité pour la suite du build **et** pour l'exécution finale.

Nuance. On le met malgré tout parce que c'est l'une des mesures de sécurité les plus rentables : sans lui, l'application tourne en `root` **dans** le conteneur, et toute faille applicative donne un root local — première marche d'une évasion. Attention à deux effets de bord : les fichiers copiés avant appartiennent à root, donc l'application ne peut pas y écrire (utilisez `COPY --chown=1000:1000`), et un utilisateur non-root ne peut pas ouvrir un port inférieur à 1024 — d'où les applications conteneurisées qui écoutent sur 8080 et non sur 80.

Exemple.

```
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY --chown=1000:1000 target/api.jar app.jar
USER 1000:1000
ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

Question 14 — « Tout dans un seul RUN »

Réponse. Il a raison pour ce qui doit être **atomique** : installation + nettoyage, téléchargement + extraction + suppression de l'archive. Séparer ces opérations laisse des fichiers dans l'image. Il a tort comme **règle générale**.

Pourquoi. Regrouper détruit la granularité du cache. Un unique `RUN` de 200 lignes est rejoué **entièrement** dès qu'un caractère change. Et une seule couche énorme est retransférée en entier au `push` ou au `pull`, alors que plusieurs couches bien découpées permettent le transfert différentiel.

Nuance. La règle « limiter les couches » date d'avant 2017, quand le nombre de couches était plafonné (127) et que le multi-stage n'existait pas. Aujourd'hui la vraie question est : **qu'est-ce qui change ensemble ?** Ce qui varie au même rythme va dans la même couche ; ce qui varie à des rythmes différents est séparé — dépendances d'un côté, code de l'autre. Le multi-stage (labo 05) a de toute façon rendu le débat largement obsolète, en supprimant du résultat final tout ce qui n'est pas nécessaire.

Exemple.

Regroupé parce qu'atomique :

RUN apk add --no-cache curl && curl -fsSL ... -o /tmp/x && tar -xf /tmp/x -C /opt && rm /tmp/x

Séparé parce que rythmes différents :

COPY pom.xml . ; RUN mvn dependency:go-offline

COPY src ./src ; RUN mvn package